

Full Stack Development with MERN Project

Documentation format

1. Introduction

- **Project Title:** Cab booking
- **Team Members:** Leela Satyasri Kanuboyina-whole project

2. Project Overview

- **Purpose:** Cab booking is a full-stack cab booking application built using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It allows users to register, book rides, track drivers in real time, make secure online payments, and manage ride history. Drivers can accept rides and update ride status, while administrators manage users, drivers, bookings, and payments.

- **Features:** User Registration & Login

- Driver Registration & Verification

- Role-Based Authentication

- Nearby Cab Search

- Fare Estimation

- Real-Time Ride Tracking

- Online Payments

- Ride Booking & Cancellation

- Booking History

- Responsive UI

- Admin Dashboard

- Driver Dashboard

• 3. Architecture Chapter

- 3.1 System Architecture

- The Cab booking – MERN Stack Cab Booking System follows a three-tier architecture consisting of the Frontend, Backend, and Database layers. This architecture ensures scalability, maintainability, and secure communication between the client and server. Users interact with the frontend application, which communicates with the backend through RESTful APIs. The backend processes requests, performs business logic, and stores or retrieves data from the MongoDB database.
- The architecture consists of the following layers:
 - Presentation Layer (Frontend) – Built using React.js.
 - Application Layer (Backend) – Developed using Node.js and Express.js.
 - Data Layer (Database) – Implemented using MongoDB and Mongoose.

- **3.2 Frontend Architecture (React.js)**

- The frontend of the Cab booking application is developed using React.js, a component-based JavaScript library that enables the creation of dynamic and responsive user interfaces. React allows the application to be divided into reusable components, improving code organization and maintainability.
- The frontend is responsible for displaying the user interface and handling user interactions such as registration, login, cab booking, ride tracking, payment, and profile management. Navigation between pages is managed using React Router, while Axios is used to communicate with the backend APIs.
- The application is organized into multiple reusable components and pages, making it easier to maintain and extend in the future.
- Frontend Components
 - Home Page
 - Login and Registration
 - User Dashboard
 - Driver Dashboard
 - Admin Dashboard
 - Cab Search
 - Booking Form

- Fare Estimation
- Ride Tracking
- Payment Page
- Booking History
- Profile Management
- Frontend Workflow
 - User opens the application.
 - React Router loads the requested page.
 - User performs an action such as booking a ride.
 - Axios sends an API request to the backend.
 - The backend processes the request.
 - The response is returned to React.
 - React updates the interface without reloading the page.
- Advantages of React Frontend
 - Component-based architecture
 - Fast rendering using Virtual DOM
 - Reusable UI components
 - Responsive user interface
 - Easy API integration
 - Efficient state management
 - Improved user experience

- **3.3 Backend Architecture (Node.js and Express.js)**

- The backend of the Cab booking system is developed using Node.js and Express.js. It acts as the middleware between the frontend and the database by handling all business logic, authentication, validation, and API requests.
- The backend exposes RESTful APIs that allow users, drivers, and administrators to interact with the system securely. Authentication is implemented using JSON Web Tokens (JWT), while passwords are encrypted using bcryptjs before being stored in the database.
- The backend follows a modular architecture consisting of configuration files, routes, controllers, middleware, and database models.
- Backend Modules
 - Authentication Module
 - User Management Module
 - Driver Management Module
 - Cab Booking Module
 - Ride Management Module
 - Payment Module
 - Admin Module
 - Notification Module
- Backend Workflow
 - The frontend sends an HTTP request.
 - Express receives the request.
 - Middleware validates authentication and input data.
 - Controllers execute business logic.
 - Mongoose interacts with MongoDB.
 - The response is returned as JSON.
 - The frontend updates the user interface.
- Backend Features

- RESTful API Development
- JWT Authentication
- Password Encryption
- Role-Based Authorization
- Input Validation
- Error Handling
- Middleware-Based Security
- Modular Project Structure

•

- **3.4 Database Architecture (MongoDB)**

- MongoDB is used as the primary NoSQL database for storing application data. The database stores information in flexible JSON-like documents, making it suitable for handling dynamic data generated by a cab booking application.
- The application uses Mongoose as the Object Data Modeling (ODM) library to define schemas, validate data, and communicate with MongoDB efficiently.
- Database Collections
- Users
 - Stores customer information.
- Fields
 - User ID
 - Full Name
 - Email
 - Phone Number
 - Password (Encrypted)
 - Role

- Profile Image
- Created Date

•

- Drivers

- Stores driver information.

- Fields

- Driver ID
- Driver Name
- Email
- Phone Number
- Driving License Number
- Vehicle Details
- Availability Status
- Current Location
- Verification Status

•

- Cabs

- Stores cab information.

- Fields

- Cab ID
- Vehicle Number
- Vehicle Model
- Vehicle Type
- Seating Capacity

- Fare per Kilometer
- Availability Status
- Driver ID

•

- Rides
- Stores ride booking information.
- Fields
 - Ride ID
 - User ID
 - Driver ID
 - Cab ID
 - Pickup Location
 - Drop Location
 - Distance
 - Estimated Fare
 - Ride Status
 - Booking Date
 - Ride Start Time
 - Ride End Time

•

- Payments
- Stores payment details.
- Fields
 - Payment ID

- Ride ID
- User ID
- Payment Method
- Transaction ID
- Payment Amount
- Payment Status
- Payment Date

•

- **3.5 Database Interactions**

- The MongoDB database interacts with the backend through Mongoose models.
- User Operations
 - Register User
 - Login User
 - Update Profile
 - View Ride History
- Driver Operations
 - Register Driver
 - Update Availability
 - Accept Ride
 - Complete Ride
- Ride Operations
 - Book Ride
 - Cancel Ride
 - Start Ride

- Complete Ride
- View Ride History
- Payment Operations
 - Create Payment
 - Verify Payment
 - Generate Receipt
 - Store Transaction Details

• **3.6 Overall System Flow**

- The overall workflow of the Cab booking system is as follows:
 - The user accesses the React application through a web browser.
 - The frontend sends API requests to the Express server.
 - Express validates the request and authenticates the user using JWT.
 - Controllers process the request and interact with MongoDB using Mongoose.
 - MongoDB stores or retrieves the required data.
 - The backend returns a JSON response.
 - React updates the interface dynamically and displays the latest information to the user.
- This architecture provides high performance, secure communication, modular development, and scalability, making Cab booking suitable for real-world cab booking applications.
-

4. Setup Instructions

4.1 Introduction

This chapter explains the software and hardware requirements, installation process, and environment configuration required to set up and run the Cab booking – MERN Stack Cab Booking System. Following these steps ensures that the application is correctly configured for development and testing.

4.2 Prerequisites

Before installing the project, ensure that the following software is installed on your system.

Software Requirements

Software	Version	Purpose
Node.js	v18 or above	JavaScript runtime environment
npm	v9 or above	Package manager
MongoDB Community Server / MongoDB Atlas	Latest	Database
Visual Studio Code	Latest	Code editor
Git	Latest	Version control
Postman	Latest	API testing
Google Chrome	Latest	Web browser
ImageKit Account	-	Image storage
Google Maps API	-	Maps and location services
Stripe/Razorpay Account	-	Payment gateway integration

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3
RAM	4 GB (8 GB Recommended)
Storage	10 GB Free Space
Operating System	Windows 10/11, Linux, or macOS
Internet Connection	Required

4.3 Project Installation

Follow the steps below to install and configure the project.

Step 1: Clone the Repository

Open Terminal or Command Prompt and execute:

```
git clone https://github.com/your-username/ucab.git
```

Move into the project directory:

```
cd cab booking
```

Step 2: Install Frontend Dependencies

Navigate to the client folder:

```
cd client
```

Install all required packages:

```
npm install
```

This installs dependencies such as:

- React.js
- React Router
- Axios
- Bootstrap or Tailwind CSS

- React Icons
 - Context API packages
-

Step 3: Install Backend Dependencies

Navigate to the server folder:

```
cd ../server
```

Install backend packages:

```
npm install
```

The backend installs dependencies including:

- Express.js
 - MongoDB (Mongoose)
 - JWT
 - bcryptjs
 - dotenv
 - cors
 - multer
 - ImageKit SDK
 - Socket.IO
 - Stripe or Razorpay SDK
-

4.4 Environment Variables

The application uses environment variables to securely store sensitive information such as database credentials, API keys, and authentication secrets.

Create a .env file inside the server directory.

Server (.env)

```
PORT=5000
```

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

IMAGEKIT_PUBLIC_KEY=your_public_key

IMAGEKIT_PRIVATE_KEY=your_private_key

IMAGEKIT_URL_ENDPOINT=https://ik.imagekit.io/your_endpoint

GOOGLE_MAPS_API_KEY=your_google_maps_key

STRIPE_SECRET_KEY=your_stripe_secret_key

Create another .env file inside the client directory.

Client (.env)

VITE_API_URL=http://localhost:5000

VITE_GOOGLE_MAPS_API_KEY=your_google_maps_key

VITE_STRIPE_PUBLISHABLE_KEY=your_publishable_key

4.5 Database Configuration

Local MongoDB

Start the MongoDB service:

mongod

MongoDB Atlas

1. Create a MongoDB Atlas account.
2. Create a new cluster.
3. Create a database user.
4. Whitelist your IP address.
5. Copy the connection string.
6. Paste it into the MONGO_URI variable.

Example:

MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/ucab

4.6 Running the Application

Start the Backend Server

Open a terminal:

```
cd server
```

```
npm run dev
```

Expected output:

Server running on port 5000

MongoDB Connected Successfully

Start the Frontend Server

Open another terminal:

```
cd client
```

```
npm run dev
```

Expected output:

Local: http://localhost:5173

Open the browser and visit:

<http://localhost:5173>

4.7 Verifying the Installation

After the application starts successfully, verify the following:

- User Registration page loads successfully.
 - Login page is accessible.
 - MongoDB connection is established.
 - Users can register and log in.
 - Available cabs are displayed.
 - Ride booking functionality works correctly.
 - Driver dashboard is accessible.
 - Admin dashboard loads successfully.
 - API endpoints respond correctly using Postman.
 - Images load properly through ImageKit.
 - Payment gateway initializes successfully.
-

4.8 Summary

By completing the above installation and configuration steps, the Cab booking MERN Stack Cab Booking System is successfully set up for development and testing. The frontend, backend, and MongoDB database communicate seamlessly through RESTful APIs, while environment variables ensure secure handling of sensitive credentials. This setup provides a reliable foundation for implementing and testing all application features.

5. Folder Structure

Cab booking/

|

|— client/

```
|   ├── public/
|   ├── src/
|   │   ├── assets/
|   │   ├── components/
|   │   ├── context/
|   │   ├── pages/
|   │   ├── services/
|   │   └── App.jsx
|   ├── package.json
|   └── .env
|
|── server/
|   ├── config/
|   ├── controllers/
|   ├── middleware/
|   ├── models/
|   ├── routes/
|   ├── sockets/
|   ├── utils/
|   ├── server.js
|   ├── package.json
|   └── .env
|
└── README.md
```

6. Running the Application

6.1 Introduction

After successfully installing all required dependencies and configuring the environment variables, the **Cab booking – MERN Stack Cab Booking System** can be executed locally by starting both the backend and frontend servers. The backend provides the RESTful APIs and database connectivity, while the frontend offers the user interface through a web browser.

6.2 Starting the Backend Server

The backend is developed using **Node.js** and **Express.js**. It handles user authentication, ride booking, driver management, payment processing, and communication with the MongoDB database.

Steps

1. Open a terminal or command prompt.
2. Navigate to the **server** directory.

`cd server`

3. Start the backend server.

If using Node.js

`npm start`

If using Nodemon (recommended during development)

`npm run dev`

Expected Output

Server is running on port 5000

MongoDB Connected Successfully

Once the server starts successfully, it will listen for API requests at:

`http://localhost:5000`

6.3 Starting the Frontend Server

The frontend is built using **React.js** and provides the graphical user interface for users, drivers, and administrators.

Steps

1. Open a new terminal window.
2. Navigate to the **client** directory.

`cd client`

3. Start the React application.

If your project uses Create React App

`npm start`

If your project uses Vite (recommended for modern React projects)

`npm run dev`

Expected Output

For Create React App:

Compiled successfully!

Local: `http://localhost:3000`

For Vite:

VITE v5.x.x ready

Local: `http://localhost:5173`

Open the displayed URL in your web browser to access the application.

6.4 Running Frontend and Backend Together

To use the application correctly, both servers must be running simultaneously.

Terminal	Directory	Command
Terminal 1 server		npm start or npm run dev
Terminal 2 client		npm start or npm run dev

6.5 Verifying the Application

After both servers are running, verify that the application is functioning correctly by performing the following checks:

- Open the application in a web browser.
- Register a new user account.
- Log in using valid credentials.
- Search for available cabs.
- Book a ride.
- View ride details and booking history.
- Log in as a driver and accept ride requests.
- Access the admin dashboard and verify user and booking management features.
- Confirm that API requests are successfully processed.
- Ensure the MongoDB database is storing and retrieving data correctly.

6.6 Application Workflow

1. Start the MongoDB database (if using a local installation).
2. Start the backend server.
3. Start the frontend server.
4. Open the application in a web browser.
5. Register or log in as a User, Driver, or Admin.
6. Use the available features such as booking rides, tracking drivers, managing bookings, and processing payments.

6.7 Summary

Running the Cab booking application requires starting both the backend and frontend servers. The backend manages business logic, authentication, and database operations, while the frontend provides an interactive user interface. Once both services are active, users can access all features of the cab booking system through their web browser.

7. API Documentation

- **Chapter 7: API Documentation**
- **7.1 Introduction**
- The Cab booking application follows a **RESTful API architecture** where the frontend communicates with the backend through HTTP requests. The backend is developed using **Node.js** and **Express.js**, while **MongoDB** stores the application data. All APIs exchange data in **JSON** format.
- The APIs are categorized into the following modules:

- Authentication APIs
- User APIs
- Driver APIs
- Ride APIs
- Payment APIs
- Admin APIs

• 7.2 Authentication APIs

• 1. Register User

- | | |
|-------------------|-------------------------|
| • Property | • Description |
| • Method | • POST |
| • Endpoint | • /api/auth/register |
| • Description | • Registers a new user. |

• Request Body

- {
- "name": "Rahul Kumar",
- "email": "rahul@gmail.com",
- "phone": "9876543210",
- "password": "password123"
- }

• Success Response (201)

- {
- "success": true,

- "message": "User registered successfully",
- "token": "JWT_TOKEN"
- }

• 2. Login User

- **Property** • **Description**
- Method • POST
- Endpoint • /api/auth/login
- Description • Authenticates a registered user.

• Request Body

- {
- "email": "rahul@gmail.com",
- "password": "password123"
- }

• Success Response

- {
- "success": true,
- "token": "JWT_TOKEN",
- "user": {
- "id": "12345",
- "name": "Rahul Kumar",
- "role": "user"
- }

- }

- **7.3 User APIs**

- **1. Get User Profile**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/users/profile
- Authentication • Required

- **Response**

- {
- "id": "12345",
- "name": "Rahul Kumar",
- "email": "rahul@gmail.com",
- "phone": "9876543210"
- }

- **2. Update User Profile**

- **Property** • **Description**
- Method • PUT
- Endpoint • /api/users/profile

- **Request Body**

- {
- "name": "Rahul Kumar",

- "phone": "9876543211"
- }
- **Response**
- {
- "success": true,
- "message": "Profile updated successfully"
- }

- **3. Get Booking History**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/users/bookings

- **Response**

- [
- {
- "rideId": "R001",
- "pickup": "Airport",
- "drop": "Railway Station",
- "fare": 350,
- "status": "Completed"
- }
-]

- **7.4 Driver APIs**

- **1. Register Driver**

- **Property • Description**

- Method • POST

- Endpoint • /api/drivers/register

- **Request Body**

- {

- "name": "Amit",

- "email": "amit@gmail.com",

- "vehicleNumber": "TS09AB1234",

- "licenseNumber": "DL123456789"

- }

- **Response**

- {

- "success": true,

- "message": "Driver registered successfully"

- }

- ---

- **2. Get Available Drivers**

- **Property • Description**

- Method • GET

- Endpoint • /api/drivers/available

- **Response**

- [

- {
- "driverName": "Amit",
- "vehicle": "Swift Dzire",
- "rating": 4.8,
- "status": "Available"
- }
-]

- **3. Update Driver Status**

- **Property** • **Description**
- Method • PUT
- Endpoint • /api/drivers/status

- **Request Body**

- {
- "status": "Busy"
- }

- **Response**

- {
- "success": true,
- "message": "Driver status updated"
- }

- **7.5 Ride APIs**

- **1. Book Ride**

- **Property** • **Description**
- Method • POST
- Endpoint • /api/rides/book

- **Request Body**

- {
- "pickup": "Airport",
- "drop": "City Center",
- "cabType": "Sedan"
- }

- **Response**

- {
- "success": true,
- "rideId": "R1001",
- "estimatedFare": 420
- }

- ---

- **2. Get Ride Details**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/rides/:id

- **Response**

- {
- "rideId": "R1001",

- "driver": "Amit",
- "cab": "Swift Dzire",
- "status": "On the Way"
- }

•

- **3. Start Ride**

- **Property** • **Description**

- Method • PUT

- Endpoint • /api/rides/start/:id

- **Response**

- {
- "success": true,
- "message": "Ride Started"
- }

•

- **4. Complete Ride**

- **Property** • **Description**

- Method • PUT

- Endpoint • /api/rides/complete/:id

- **Response**

- {
- "success": true,
- "message": "Ride Completed"

- }

-

- **5. Cancel Ride**

- **Property** • **Description**

- Method • DELETE

- Endpoint • /api/rides/cancel/:id

- **Response**

- {

- "success": true,

- "message": "Ride Cancelled"

- }

-

- **7.6 Payment APIs**

- **1. Create Payment**

- **Property** • **Description**

- Method • POST

- Endpoint • /api/payments/create

- **Request Body**

- {

- "rideId": "R1001",

- "amount": 420,

- "method": "Card"

- }

- **Response**

- {
- "success": true,
- "paymentId": "P5001",
- "status": "Pending"
- }

- **2. Verify Payment**

- **Property** • **Description**
- Method • POST
- Endpoint • /api/payments/verify

- **Response**

- {
- "success": true,
- "message": "Payment Successful"
- }

- **7.7 Admin APIs**

- **1. Get All Users**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/admin/users

- **Response**

- [
- {
- "name": "Rahul",
- "email": "rahul@gmail.com",
- "role": "User"
- }
-]

•

- **2. Get All Drivers**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/admin/drivers

- **Response**

- [
- {
- "driverName": "Amit",
- "status": "Available"
- }
-]

•

- **3. Get All Rides**

- **Property** • **Description**
- Method • GET

- **Property** • **Description**
- Endpoint • /api/admin/rides

- **Response**

- [
 - {
 - "rideId": "R1001",
 - "user": "Rahul",
 - "driver": "Amit",
 - "status": "Completed"
 - }
-]

- ---

- **4. Dashboard Statistics**

- **Property** • **Description**
- Method • GET
- Endpoint • /api/admin/dashboard

- **Response**

- {
 - "totalUsers": 520,
 - "totalDrivers": 110,
 - "totalRides": 2480,
 - "completedRides": 2315,
 - "totalRevenue": 425000

- }

- **7.8 API Security**

- The Cab booking backend implements the following security mechanisms:

- JWT-based authentication for protected routes.
- Password hashing using **bcryptjs** before storing credentials.
- Role-based authorization for **User**, **Driver**, and **Admin**.
- Input validation to prevent invalid or malicious requests.
- CORS configuration to control cross-origin access.
- Environment variables for securely storing API keys and secrets.

- **7.9 Summary**

- The Cab booking backend exposes a comprehensive set of RESTful APIs that support user authentication, ride booking, driver management, payment processing, and administrative operations. These APIs enable secure and efficient communication between the React frontend and the Node.js backend while ensuring scalability, maintainability, and reliable data management.

- .

8.Authentication

8.1 Introduction

Authentication and authorization are essential components of the Cab booking – MERN Stack Cab Booking System. They ensure that only authenticated users can access the application and that users can perform only the actions permitted by their assigned roles. The system uses JSON Web Tokens (JWT) for authentication and bcryptjs for password encryption, providing a secure and scalable authentication mechanism.

8.2 Authentication

Authentication is the process of verifying the identity of a user before granting access to the application.

The Cab booking system supports three types of users:

- User (Customer)
- Driver
- Administrator (Admin)

Each user must register with valid credentials and log in before accessing protected resources.

Authentication Process

1. The user registers by providing personal details such as name, email, phone number, and password.
 2. The password is encrypted using bcryptjs before being stored in the MongoDB database.
 3. During login, the entered password is compared with the encrypted password stored in the database.
 4. If the credentials are valid, the server generates a JSON Web Token (JWT).
 5. The JWT is sent to the client and stored securely.
 6. For every protected API request, the client sends the JWT in the request header.
 7. The server validates the token before processing the request.
-

8.3 User Registration

During registration, the following information is collected:

- Full Name
- Email Address
- Phone Number
- Password
- User Role

Registration Workflow

1. User fills out the registration form.
 2. Backend validates the input.
 3. Password is hashed using bcryptjs.
 4. User details are stored in MongoDB.
 5. A success response is returned.
-

8.4 User Login

After successful registration, users can log in using their email and password.

Login Workflow

1. User enters email and password.
 2. Server searches for the user in MongoDB.
 3. Password is verified using bcryptjs.
 4. JWT token is generated.
 5. Token is returned to the client.
 6. User gains access to protected pages.
-

8.5 Password Encryption

The Cab booking system uses bcryptjs to securely hash passwords before storing them in the database.

Why Password Hashing?

- Prevents storing plain-text passwords.
- Protects user accounts in case of a database breach.
- Adds a unique salt to each password.
- Makes passwords difficult to reverse using brute-force attacks.

Password Storage Example

Instead of storing:

password123

The database stores a hashed value such as:

\$2b\$10\$Gk5lQ8gV1A4Q4m0z0WbAuFh9P7N2M4YtPzjK2cB5YQdT8M1L9xAa

8.6 JSON Web Token (JWT)

JWT is used for secure user authentication after login.

A JWT consists of three parts:

- Header
- Payload
- Signature

Example:

xxxxx.yyyyy.zzzzz

The payload typically contains:

- User ID
- Email
- User Role
- Token Expiration Time

The server signs the token using a secret key defined in the .env file.

Example:

JWT_SECRET=your_secret_key

8.7 Token-Based Authentication

Once a user logs in successfully, the backend generates a JWT.

The client stores the token securely (for example, in an HTTP-only cookie or browser storage, depending on the implementation).

For every protected API request, the token is included in the HTTP Authorization header.

Example Request Header

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

The server verifies the token before allowing access to protected resources.

8.8 Authorization

Authorization determines what actions an authenticated user is allowed to perform.

The Cab booking application implements Role-Based Access Control (RBAC) with three roles.

User Permissions

Users can:

- Register and log in.
- Search available cabs.
- Book rides.
- Cancel bookings.
- View ride history.
- Update profile information.
- Make online payments.

Users cannot:

- Access the admin dashboard.
- Manage drivers.
- View other users' bookings.

Driver Permissions

Drivers can:

- Register and log in.
- View assigned ride requests.
- Accept or reject ride requests.
- Update ride status.
- Manage availability.
- View earnings and ride history.

Drivers cannot:

- Manage users.
 - Access administrator functions.
-

Admin Permissions

Administrators can:

- Manage users.
 - Manage drivers.
 - Verify driver accounts.
 - Manage bookings.
 - Monitor payments.
 - View reports and analytics.
 - Access the complete administrative dashboard.
-

8.9 Authentication Middleware

The backend uses Express middleware to protect private routes.

The middleware performs the following operations:

1. Reads the JWT from the Authorization header.

2. Verifies the token using the secret key.
3. Decodes the user information.
4. Attaches the authenticated user to the request object.
5. Passes control to the next middleware or controller.

If the token is invalid or expired, the request is rejected with an Unauthorized response.

8.10 Session Management

Cab booking follows a stateless authentication model using JWT.

Unlike traditional server-side sessions:

- No session data is stored on the server.
- Every request contains the authentication token.
- The server validates the token independently.
- This approach improves scalability and simplifies deployment.

Token expiration can be configured (for example, one hour or one day), after which the user must log in again or refresh the token, depending on the implementation.

8.11 Authentication Flow

The authentication process follows these steps:

1. User submits registration details.
2. Password is hashed using bcryptjs.
3. User information is stored in MongoDB.
4. User logs in with email and password.
5. Backend validates credentials.
6. JWT token is generated.
7. Client stores the token securely.
8. Client sends the token with each protected request.

9. Middleware verifies the token.
10. Authorized users access the requested resources based on their role.

8.12 Security Best Practices

The Cab booking system follows several security practices:

- Passwords are hashed using bcryptjs.
- JWT is used for secure authentication.
- Protected routes require a valid token.
- Role-based authorization prevents unauthorized access.
- Environment variables protect sensitive configuration values.
- Input validation helps prevent invalid or malicious requests.
- CORS is configured to restrict unauthorized cross-origin requests.
- Sensitive information such as passwords is never returned in API responses.

8.13 Advantages of JWT Authentication

- Stateless and scalable architecture.
- Faster authentication without server-side session storage.
- Reduced server memory usage.
- Easy integration with React applications.
- Secure communication using signed tokens.
- Suitable for RESTful APIs and distributed systems.

8.14 Summary

The Cab booking MERN Stack Cab Booking System uses a secure authentication and authorization mechanism based on JWT and bcryptjs. Passwords are encrypted before storage, authenticated users receive signed tokens after successful login, and role-based authorization ensures that users, drivers, and administrators can access only the resources appropriate to their permissions. This approach enhances application security, scalability, and overall reliability.

9. User Interface

- <https://car-rental-pi-amber.vercel.app/>

10. Testing

- **10.1 Introduction**

- Testing is an essential phase of software development that ensures the **Cab Booking System – MERN Stack Web Application** functions correctly, securely, and efficiently. The primary objective of testing is to identify and resolve defects before deployment, ensuring a reliable and user-friendly application. The testing process verifies that each module works independently and that all modules integrate seamlessly.

- ---

- **10.2 Testing Strategy**

- The **Cab Booking System** follows a structured testing strategy consisting of multiple levels of testing to ensure the quality of the application.
- The testing strategy includes:
 - Unit Testing
 - Integration Testing
 - System Testing
 - Functional Testing
 - User Acceptance Testing (UAT)
 - Performance Testing
 - Security Testing
 - Responsive Testing
- Each testing level validates different aspects of the application, from individual components to the complete system.

- ---

- **10.3 Unit Testing**

- Unit testing verifies individual functions, components, and modules independently.

- **Objective**

- Verify that each module functions correctly.
- Detect coding errors early.
- Improve software reliability.

- **Modules Tested**

- User Authentication
- Driver Registration
- Ride Booking
- Payment Processing
- Profile Management
- Fare Calculation

- **Expected Outcome**

- Each module should produce the expected output for valid inputs and handle invalid inputs appropriately.

- ---

- **10.4 Integration Testing**

- Integration testing ensures that multiple modules work together correctly.

- **Modules Integrated**

- React Frontend ↔ Express Backend
- Backend ↔ MongoDB Database
- Booking Module ↔ Payment Module
- Authentication Module ↔ Protected Routes
- Driver Module ↔ Ride Management

- **Objective**

- Verify data flow between modules.
- Identify interface issues.
- Ensure API communication is functioning correctly.

- ---

- **10.5 System Testing**

- System testing validates the complete application as a whole.

- **Features Tested**

- User Registration
- Login Authentication
- Driver Registration
- Ride Booking
- Live Ride Tracking
- Payment Integration
- Booking History
- Admin Dashboard

- **Expected Result**

- All modules should operate together without errors, providing a complete and seamless user experience.

- ---

- **10.6 Functional Testing**

- Functional testing verifies that every feature performs according to the specified requirements.

- **Functional Test Cases**

- User can register successfully.

- User can log in with valid credentials.
 - Driver can accept ride requests.
 - User can book a cab.
 - Fare estimation is calculated correctly.
 - Payment is processed successfully.
 - Ride history is displayed accurately.
 - Admin can manage users and drivers.
-
- **10.7 User Acceptance Testing (UAT)**
 - User Acceptance Testing is performed to ensure the system satisfies end-user requirements.
 - **Objectives**
 - Validate usability.
 - Confirm business requirements.
 - Verify overall user experience.
 - **Participants**
 - Customers
 - Drivers
 - System Administrator
 - **Expected Outcome**
 - The application should meet all functional and usability expectations before deployment.
-
- **10.8 Performance Testing**

- Performance testing evaluates the responsiveness and stability of the application under different workloads.

- **Parameters Tested**

- API Response Time
- Database Query Performance
- Concurrent User Requests
- Page Loading Speed
- Ride Search Performance

- **Expected Results**

- Fast response time.
- Stable performance under multiple user requests.
- Efficient database operations.

- ---

- **10.9 Security Testing**

- Security testing ensures that the application protects user data and prevents unauthorized access.

- **Security Features Tested**

- JWT Authentication
- Password Encryption (bcryptjs)
- Role-Based Authorization
- Protected API Routes
- Input Validation
- Cross-Origin Resource Sharing (CORS)
- Environment Variable Protection

- **Expected Results**

- Unauthorized users cannot access protected resources.
- Passwords remain encrypted.
- Tokens are validated correctly.
- Sensitive information is protected.

•

- **10.10 Responsive Testing**

- Responsive testing verifies that the application works correctly on different devices and screen sizes.

- **Devices Tested**

- Desktop
- Laptop
- Tablet
- Android Smartphones
- iPhone

- **Browsers Tested**

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari

- **Expected Results**

- Responsive layouts.
- Proper alignment of UI components.
- Consistent functionality across browsers.

•

- **10.11 Testing Tools Used**

- The following tools were used during testing:

• Tool	• Purpose
• Postman	• Testing RESTful APIs
• MongoDB Compass	• Database verification
• Google Chrome DevTools	• UI inspection and debugging
• Visual Studio Code	• Code debugging and development
• Node.js	• Backend execution
• npm	• Dependency management
• Git & GitHub	• Version control
• React Developer Tools	• Inspect React components
• Browser Console	• JavaScript debugging

- ---

- **10.12 Sample Test Cases**

• Test Case ID	• Test Scenario	• Expected Result	• Status
• TC-01	• User Registration	• User account created successfully	• Pass
• TC-02	• User Login	• JWT token generated	• Pass
• TC-03	• Search Available Cabs	• List of available cabs displayed	• Pass
• TC-04	• Book Ride	• Ride booked successfully	• Pass

• Test Case ID	• Test Scenario	• Expected Result	• Status
• TC-05	• Driver Accepts Ride	• Ride status updated	• Pass
• TC-06	• Online Payment	• Payment completed successfully	• Pass
• TC-07	• View Booking History	• Booking records displayed	• Pass
• TC-08	• Admin Login	• Admin dashboard accessible	• Pass
• TC-09	• Invalid Login	• Error message displayed	• Pass
• TC-10	• Unauthorized API Access	• Access denied (401 Unauthorized)	• Pass

• 10.13 Bug Tracking and Error Handling

• During testing, identified issues were logged and resolved using the following process:

- Identify the defect.
- Reproduce the issue.
- Analyze the root cause.
- Fix the issue in the code.
- Perform regression testing.
- Verify successful resolution.

• This process helped improve application reliability and maintainability.

• 10.14 Summary

- The **Cab Booking System – MERN Stack Web Application** was thoroughly tested using multiple testing strategies, including unit, integration, system, functional, performance, security, and responsive testing. Tools such as **Postman**, **MongoDB Compass**, **Google Chrome Developer Tools**, and **Visual Studio Code** were used to validate application functionality, API responses, database interactions, and user interface behavior. The testing process confirmed that the application meets its functional requirements, provides secure access, and delivers a stable and responsive user experience across different devices and browsers.
-

11. Screenshots or Demo

- <https://car-rental-pi-amber.vercel.app/>

12. Known Issues

- **12.1 Introduction**
- Although the **Cab Booking System – MERN Stack Web Application** has been developed and tested to provide a reliable and secure user experience, certain limitations and known issues may still exist. These issues do not significantly affect the core functionality of the application but may require improvements in future releases. Identifying these known issues helps developers maintain the system more effectively and informs users about current limitations.
- ---
- **12.2 Known Issues**
- The following are the known issues identified during development and testing.
- **1. Real-Time Driver Tracking**
- **Issue:**
The application provides ride status updates; however, real-time GPS tracking may experience slight delays due to network latency or API limitations.
- **Impact:**
Driver location updates may not always appear instantly.
- **Suggested Improvement:**
Implement WebSockets (Socket.IO) or Firebase Realtime Database for faster live location synchronization.
- ---
- **2. Payment Gateway in Development Mode**

- **Issue:**
The payment module currently uses test credentials during development.
- **Impact:**
Actual financial transactions cannot be completed in the development environment.
- **Suggested Improvement:**
Configure production API keys and enable a live payment gateway before deployment.

•

- **3. Internet Dependency**

- **Issue:**
The application requires an active internet connection to communicate with the backend server and MongoDB database.
- **Impact:**
Users cannot access online services when offline.
- **Suggested Improvement:**
Implement offline support using Progressive Web App (PWA) technologies where appropriate.

•

- **4. Browser Compatibility**

- **Issue:**
The application is optimized for modern web browsers.
- **Impact:**
Some interface elements may not function correctly in older browsers.
- **Suggested Improvement:**
Use modern JavaScript transpilation and conduct additional cross-browser compatibility testing.

•

- **5. Email Notifications**

- **Issue:**
Automatic email notifications for booking confirmation or password reset may not be fully configured in the development version.

- **Impact:**
Users may not receive email confirmations.
- **Suggested Improvement:**
Integrate a production email service such as Nodemailer with SMTP or a cloud email provider.

- **6. File Upload Restrictions**

- **Issue:**
Only supported image formats are accepted for uploading profile pictures or vehicle images.
- **Impact:**
Unsupported file formats are rejected.
- **Suggested Improvement:**
Provide clearer validation messages and automatic image optimization before upload.

- **7. Server Performance Under Heavy Load**

- **Issue:**
Application response time may increase when many users access the system simultaneously.
- **Impact:**
API responses and page loading may become slower during peak usage.
- **Suggested Improvement:**
Implement caching, load balancing, and database query optimization to improve scalability.

- **8. Session Expiration**

- **Issue:**
JWT authentication tokens expire after a predefined period.
- **Impact:**
Users are required to log in again after the token expires.

- **Suggested Improvement:**
Implement secure refresh tokens or silent token renewal to improve user experience.
- ---
- **9. Location Accuracy**
- **Issue:**
Cab availability and estimated routes depend on the accuracy of location services provided by the user's device.
- **Impact:**
Incorrect location data may affect ride matching and fare estimation.
- **Suggested Improvement:**
Enable continuous location updates and improve location validation.
- ---
- **10. Limited Notification System**
- **Issue:**
The current version provides only basic notifications within the application.
- **Impact:**
Users may not receive instant alerts for booking updates when the application is closed.
- **Suggested Improvement:**
Integrate push notification services such as Firebase Cloud Messaging (FCM) for real-time notifications.
- ---
- **12.3 Limitations**
- The current version of the Cab Booking System has the following limitations:
 - Offline functionality is not supported.
 - Live GPS tracking depends on third-party services.
 - Real payment processing requires production credentials.
 - Performance may vary depending on internet speed and server resources.

- Advanced analytics and AI-based ride recommendations are not included in the current version.

- **12.4 Future Resolution Plan**

- The following enhancements are planned to address the identified issues:

- Improve real-time tracking using Socket.IO.
- Integrate a production-ready payment gateway.
- Add push notifications for ride updates.
- Implement refresh token-based authentication.
- Optimize MongoDB queries and backend performance.
- Enhance browser compatibility and accessibility.
- Add offline support using Progressive Web App (PWA) features.
- Introduce AI-based driver matching and route optimization.

- **12.5 Summary**

- The identified issues and limitations do not prevent the **Cab Booking System – MERN Stack Web Application** from performing its core functions effectively. Most of these issues are related to advanced features, scalability, or production deployment and can be addressed in future versions. Continuous testing, monitoring, and optimization will further enhance the application's performance, security, and user experience.

-

13. Future Enhancements

Chapter 13: Future Enhancements

13.1 Introduction

The **Cab Booking System – MERN Stack Web Application** has been designed with a modular and scalable architecture, making it easy to introduce new features and improvements in future versions. As technology and user expectations continue to evolve, additional functionalities can

be incorporated to enhance the application's performance, security, and overall user experience. The following enhancements are proposed for future development.

13.2 Proposed Future Enhancements

1. Real-Time Ride Tracking

Description:

Integrate advanced real-time GPS tracking using **Socket.IO**, **Google Maps API**, or **Mapbox** to provide live updates of the driver's location.

Benefits

- Live driver location updates.
 - Accurate estimated arrival time (ETA).
 - Improved ride monitoring for users.
-

2. Ride Scheduling

Description:

Allow users to schedule rides in advance by selecting a future date and time.

Benefits

- Convenient ride planning.
 - Increased customer satisfaction.
 - Better driver availability management.
-

3. AI-Based Driver Matching

Description:

Implement Artificial Intelligence (AI) algorithms to automatically assign the nearest and most suitable driver based on location, traffic conditions, driver ratings, and availability.

Benefits

- Faster ride allocation.
- Reduced waiting time.

- Improved operational efficiency.
-

4. Ride Sharing (Carpooling)

Description:

Introduce a ride-sharing feature that allows multiple users traveling in the same direction to share a cab.

Benefits

- Reduced travel costs.
 - Lower traffic congestion.
 - Environmentally friendly transportation.
-

5. Online Wallet Integration

Description:

Provide an in-app digital wallet that enables users to store funds, receive cashback, and make quick payments.

Benefits

- Faster payment process.
 - Improved user convenience.
 - Loyalty rewards and promotional offers.
-

6. Push Notifications

Description:

Integrate **Firestore Cloud Messaging (FCM)** to send real-time notifications for booking confirmations, driver arrival updates, ride completion, and promotional offers.

Benefits

- Instant ride updates.
- Better communication.
- Enhanced user engagement.

7. Voice-Based Cab Booking

Description:

Enable users to book rides using voice commands through speech recognition technology.

Benefits

- Hands-free operation.
- Improved accessibility.
- Faster booking process.

8. SOS Emergency Feature

Description:

Include an emergency SOS button that allows users to instantly notify emergency contacts or local authorities during emergencies.

Benefits

- Improved passenger safety.
- Quick emergency response.
- Increased user confidence.

9. Driver Rating and Feedback System

Description:

Allow users to rate drivers and provide feedback after completing each ride.

Benefits

- Improved service quality.
- Driver performance evaluation.
- Increased customer satisfaction.

10. Multi-Language Support

Description:

Support multiple regional and international languages to make the application accessible to a wider audience.

Benefits

- Better user experience.
 - Increased accessibility.
 - Wider market reach.
-

11. Dark Mode**Description:**

Provide users with an option to switch between light and dark themes.

Benefits

- Reduced eye strain.
 - Improved readability.
 - Personalized user experience.
-

12. Advanced Analytics Dashboard**Description:**

Enhance the administrator dashboard with detailed reports and graphical analytics.

Features

- Daily and monthly bookings.
- Revenue analysis.
- Driver performance reports.
- Customer growth statistics.
- Peak booking hours.

Benefits

- Better business insights.

- Data-driven decision-making.
 - Improved operational planning.
-

13. Electric Vehicle (EV) Booking

Description:

Introduce an option for users to book electric vehicles as part of the fleet.

Benefits

- Environmentally friendly transportation.
 - Reduced carbon emissions.
 - Support for sustainable mobility.
-

14. Coupon and Discount Management

Description:

Implement promotional offers, referral rewards, and discount coupons.

Benefits

- Increased customer retention.
 - Higher user engagement.
 - More bookings during promotional campaigns.
-

15. Progressive Web Application (PWA)

Description:

Convert the application into a Progressive Web Application to enable offline access and installation on mobile devices.

Benefits

- Faster loading speed.
- Offline functionality for selected features.
- Improved mobile user experience.

13.3 Future Technology Integration

The following technologies can be integrated in future releases to improve the application's capabilities:

Technology	Purpose
Socket.IO	Real-time communication
Firebase Cloud Messaging	Push notifications
Google Maps API	Live navigation and route optimization
Mapbox	Interactive maps and location services
Machine Learning	Driver recommendation and demand prediction
Docker	Containerized deployment
Kubernetes	Scalable application deployment
Redis	Caching and improved performance
Elasticsearch	Faster ride and location search
CI/CD Pipelines	Automated testing and deployment

13.4 Expected Benefits

The proposed enhancements will provide the following advantages:

- Improved user experience.
- Faster ride booking and allocation.
- Better security and passenger safety.
- Enhanced system performance and scalability.
- Increased customer satisfaction.
- Reduced operational costs.
- Smarter decision-making through analytics.

- Greater accessibility and convenience.

13.5 Summary

The **Cab Booking System – MERN Stack Web Application** provides a strong foundation for future growth and innovation. By incorporating advanced technologies such as Artificial Intelligence, real-time communication, digital wallets, ride sharing, and cloud-based services, the application can evolve into a comprehensive and intelligent transportation platform. These enhancements will improve usability, efficiency, scalability, and overall service quality, making the system suitable for real-world commercial deployment.